Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL603**	**Course Name:**	**SEPM**

Name of Student:	**Shravani S. Raut**
Roll No.:	**40**
Experiment No.:	**06**
Title of the Experiment:	**To understand Jenkins Master-Slave Architecture and scale your Jenkins standalone implementation by implementing slave nodes.**
Date of Performance:	**18/02/2026**
Date of Submission:	**25/02/2026**

Evaluation

Performance Indicator	**Max. Marks**	**Marks Obtained**
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	**Exceed Expectations (EE)**	**Meet Expectations (ME)**	**Meet Expect Below Expectations (BE)**
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms Rujuta Vartak
Signature:
Date:

Expriment No.06

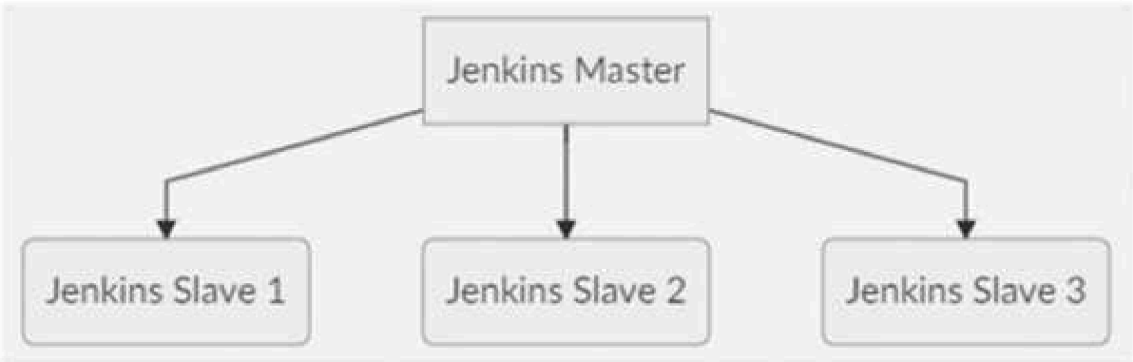
Aim: To understand Jenkins Master-Slave Architecture and scale your Jenkins standalone implementation by implementing slave nodes.

Objective: The objective of understanding Jenkins Master-Slave architecture is to comprehend how to scale Jenkins infrastructure by implementing slave nodes, thereby distributing the workload and enhancing the performance and resilience of the Jenkins CI/CD system

Theory:

A Jenkins master comes with the basic installation of Jenkins, and in this configuration, the master handles all the tasks for your build system. You are working on multiple projects, you may run multiple jobs on each project. Some projects need to run on some nodes, and in this process, we need to configure slaves. Jenkins slaves connect to the Jenkins master using the Java Network Launch Protocol.

Jenkins Master and Slave Architecture



The Jenkins master acts to schedule the jobs, assign slaves, and send builds slaves to execute the jobs. It will also monitor the slave state (offline or online) and get back the build result responses from slaves and the display build results on the console output. The workload of building jobs is delegated to multiple slaves.

Steps:

1. Click on Manage Jenkins in the left corner on the Jenkins dashboard.
2. Scroll down, Click on Manage Nodes and clouds.

3. Select New Node and enter the name of the node in the Node Name field.

Select **New Node** and enter the name of the node in the Node Name field.

4. Select Permanent Agent and click the OK button. Initially, you will get only one option, "Permanent Agent." Once you have one or more slaves you will get the "Copy Existing Node" option.

Select **Permanent Agent** and click the **OK** button. Initially, you will get only one option, "Permanent Agent." Once you have one or more slaves you will get the "Copy Existing Node" option.

In the above screen shot, Parallel_Agent_01 was Created and currently it is in offline mode.

5. Click on configure, Provide the details.

1. Name -Parallel_Agent_01.

2. Number of executors- 5.

3. Remote root directory-We have to provide a Jenkins path.

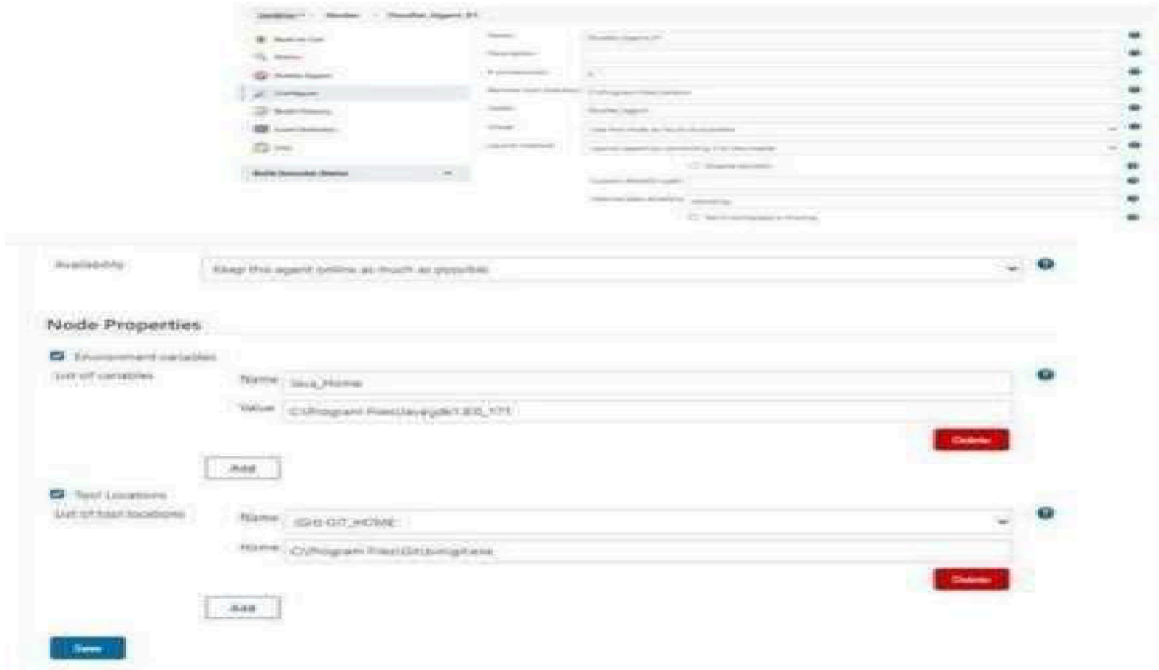
4. Labels-Parallel_Agent.

5. Launch method-Launch agent by connecting it to the master.

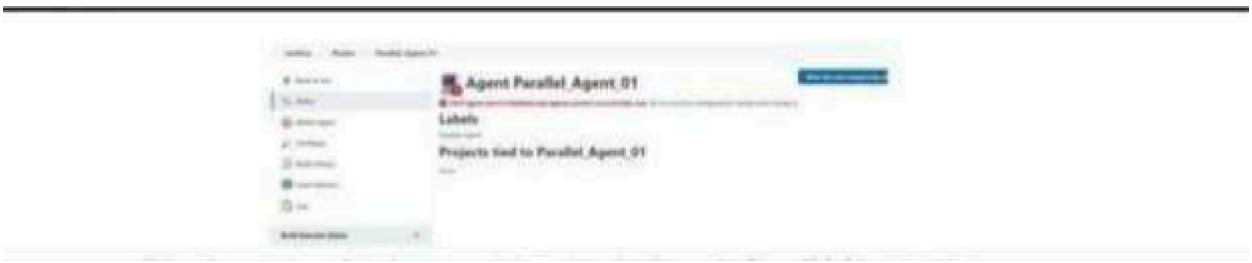
• Check Environment variables Provide the Java path

• Check Tool Locations

Provide the Git path and click on save button.

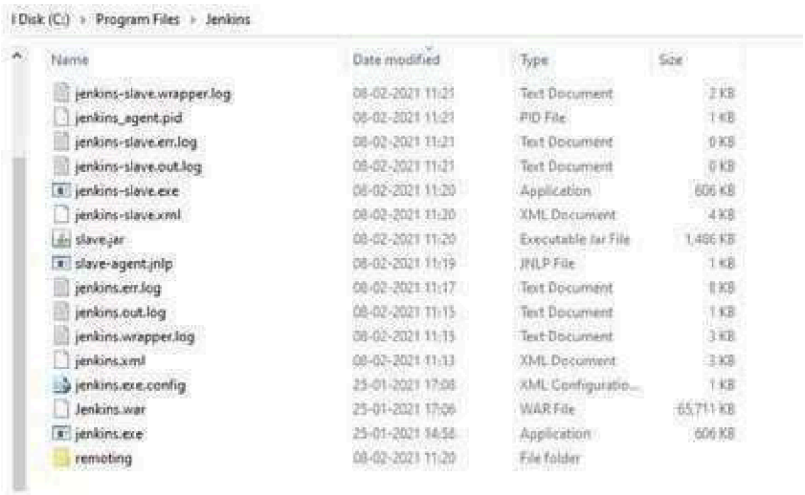


7. Click on Go to the security configuration screen and change it. It will redirect to Configure Global Security → Agents > click on Fixed radio button port: 49187 and click on save Button. Go back to Nodes settings.



1. Click on Launch button, it will download the launch agent in your system.

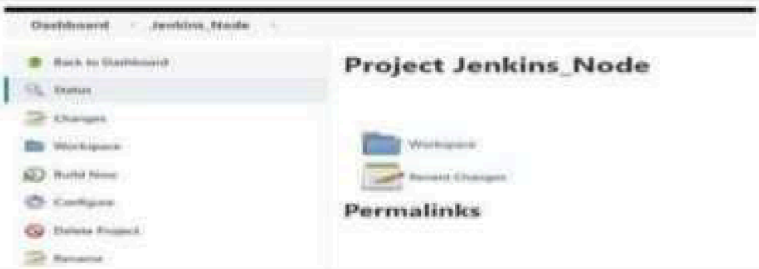
3. Double Click on jenkins - slave.exe.



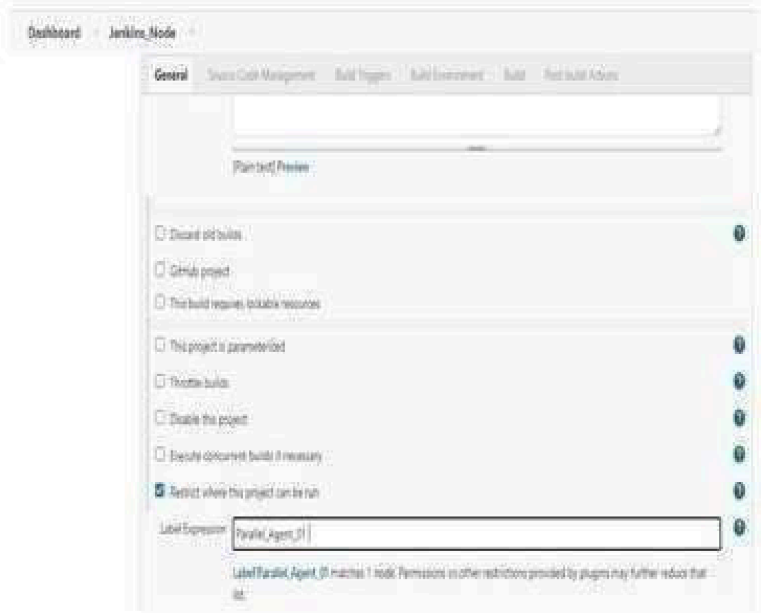
4. Run the launch agent, click on the run button and it will show connected.
Menu, menu, select the install as service and click yes button. Once it is done, refresh the page.

9. we can see the Build executors. One is master and other is Parallel_agent_01 In Master node, we can see the number of executors as 2.

- In Parallel_agent_01, we can see the number of executors as 5.
- Go to build job -> configure.



In the General tab, check on Restrict where this project can be run.
- In Label Expression, we have to select the node name where
we need to execute the build job.



Conclusion:

Q1. How does Jenkins communicate between master and slave?

Jenkins follows a Master-Slave architecture for distributed builds. The Master controls the build process, while Slaves (agents) execute jobs. Jenkins communicates between Master and Slave using the following methods:

1. SSH (Secure Shell) Connection

o The Jenkins Master connects to Slaves using SSH (mainly for Linux-based Slaves).

o The Master issues commands and retrieves build results.

2. JNLP (Java Network Launch Protocol)

o For Windows or cloud-based Slaves, JNLP agents connect to the Master.

o The Slave machine runs the JNLP agent, which registers itself with the Master.

3. WebSockets (for Cloud Agents)

o Jenkins uses WebSockets for agents behind firewalls or proxies.

4. Cloud Provider APIs (Kubernetes, AWS, Azure, etc.)

o Jenkins dynamically provisions agents using Kubernetes, Docker, or cloud platforms.

o Jenkins Master communicates with cloud-based agents via API calls.

5. Message Queue Mechanism

o Jenkins Master assigns jobs and waits for responses from Slaves.

Process Flow:

1. The Master assigns a job to a Slave.
2. The Slave executes the job and reports back the status.
3. The Master collects logs, test results, and artifacts

Q2. How many slaves can be connected to Jenkins master?

- Jenkins does not have a hard limit on the number of Slaves.
- The number of Slaves depends on:
  - o Hardware resources of the Master (CPU, RAM, network).
  - o Network bandwidth (for fast communication).
  - o Type of builds (heavy builds require more Slaves).
  - o Jenkins configuration (optimized job scheduling and agent management).

Typical Configurations:

- Small-scale CI/CD setup: 2-5 Slaves
- Medium-scale setup: 10-50 Slaves
- Enterprise setup: 100+ Slaves (using Kubernetes, cloud-based autoscaling)